CS 6025 **Introduction Parallel Processing- Assignment 3**

Jayashree Janani Purushothaman Student #: 3772545

A) Design Summary

- Description of adaptive integration

The program uses Adaptive Simpson's Rule to compute the integral over $[0, 1]$. For each interval, it first computes a Simpson estimate over the whole interval and then a refined estimate by splitting the interval into two halves. If the difference is within the required tolerance, the interval is accepted; otherwise, the interval is split recursively and the same process is repeated. This makes the method efficient for irregular functions because more work is focused only where the function changes rapidly.

- MPI dynamic design

The program uses Adaptive Simpson's Rule to compute the integral over $[0, 1]$. For each interval, it first computes a Simpson estimate over the whole interval and then a refined estimate by splitting the interval into two halves. If the difference is within the required tolerance, the interval is accepted; otherwise, the interval is split recursively and the same process is repeated. This makes the method efficient for irregular functions because more work is focused only where the function changes rapidly.

- Hybrid design

Mode 1 reuses the Assignment 2 dynamic master–worker implementation. Rank 0 acts as the master and maintains a task queue, while worker processes request work, process intervals, and send results back. Workers can also generate additional sub-intervals and return them to the master, which allows dynamic redistribution of work. This design provides good load balancing for irregular adaptive workloads because busy regions of the domain can be redistributed across processes.

- OpenMP strategy

Mode 2 implements hybrid MPI + OpenMP. In this mode, the master splits $[0, 1]$into Kcoarse intervals and distributes them statically among MPI processes. Each process then computes its assigned intervals locally. Unlike Mode 1, workers do not send newly created sub-intervals back to the master. When a local interval is split into two halves, both halves remain within the same process. This removes global dynamic load balancing and allows the assignment to study whether local thread-level parallelism can compensate for it.

B) Results Tables

- *Runtime tables*

Mode	Threads	Process	Function	Tol	Time
0		1	0	1e-6	0.000029
0		1	1	1e-6	0.000007
0		1	2	1e-6	0.000026
1		2	0	1e-6	0.000376
1		2	1	1e-6	0.000278
1		2	2	1e-6	0.001615
1		4	0	1e-6	0.000484
1		4	1	1e-6	0.000933
1		4	2	1e-6	0.000686
2	1	2	0	1e-6	0.000212

2	1	2	1	1e-6	0.000105
2	1	2	2	1e-6	0.000600
2	2	2	0	1e-6	0.000323
2	2	2	1	1e-6	0.000607
2	2	2	2	1e-6	0.000906
2	4	2	0	1e-6	0.001860
2	4	2	1	1e-6	0.002256
2	4	2	2	1e-6	0.004471
2	1	4	0	1e-6	0.000670
2	1	4	1	1e-6	0.000149
2	1	4	2	1e-6	0.000372
2	2	4	0	1e-6	0.154288
2	2	4	1	1e-6	0.110666
2	2	4	2	1e-6	0.158478
2	4	4	0	1e-6	0.585603
2	4	4	1	1e-6	0.600152
2	4	4	2	1e-6	0.597990
0		1	0	1e-8	0.000040
0		1	1	1e-8	0.000015
0		1	2	1e-8	0.000033
1		2	0	1e-8	0.000205
1		2	1	1e-8	0.000196
1		2	2	1e-8	0.001071
1		4	0	1e-8	0.000416
1		4	1	1e-8	0.000968
1		4	2	1e-8	0.001310
2	1	2	0	1e-8	0.000132
2	1	2	1	1e-8	0.000102
2	1	2	2	1e-8	0.002860
2	2	2	0	1e-8	0.000291
2	2	2	1	1e-8	0.000316
2	2	2	2	1e-8	0.004119
2	4	2	0	1e-8	0.002888
2	4	2	1	1e-8	0.003109
2	4	2	2	1e-8	0.006162
2	1	4	0	1e-8	0.000122
2	1	4	1	1e-8	0.000217
2	1	4	2	1e-8	0.001414
2	2	4	0	1e-8	0.156988
2	2	4	1	1e-8	0.157565
2	2	4	2	1e-8	0.181511
2	4	4	0	1e-8	0.595575
2	4	4	1	1e-8	0.586410
2	4	4	2	1e-8	0.586549

CS 6025 **Introduction Parallel Processing- Assignment 3**

Jayashree Janani Purushothaman Student #: 3772545

- *Speedup and efficiency*

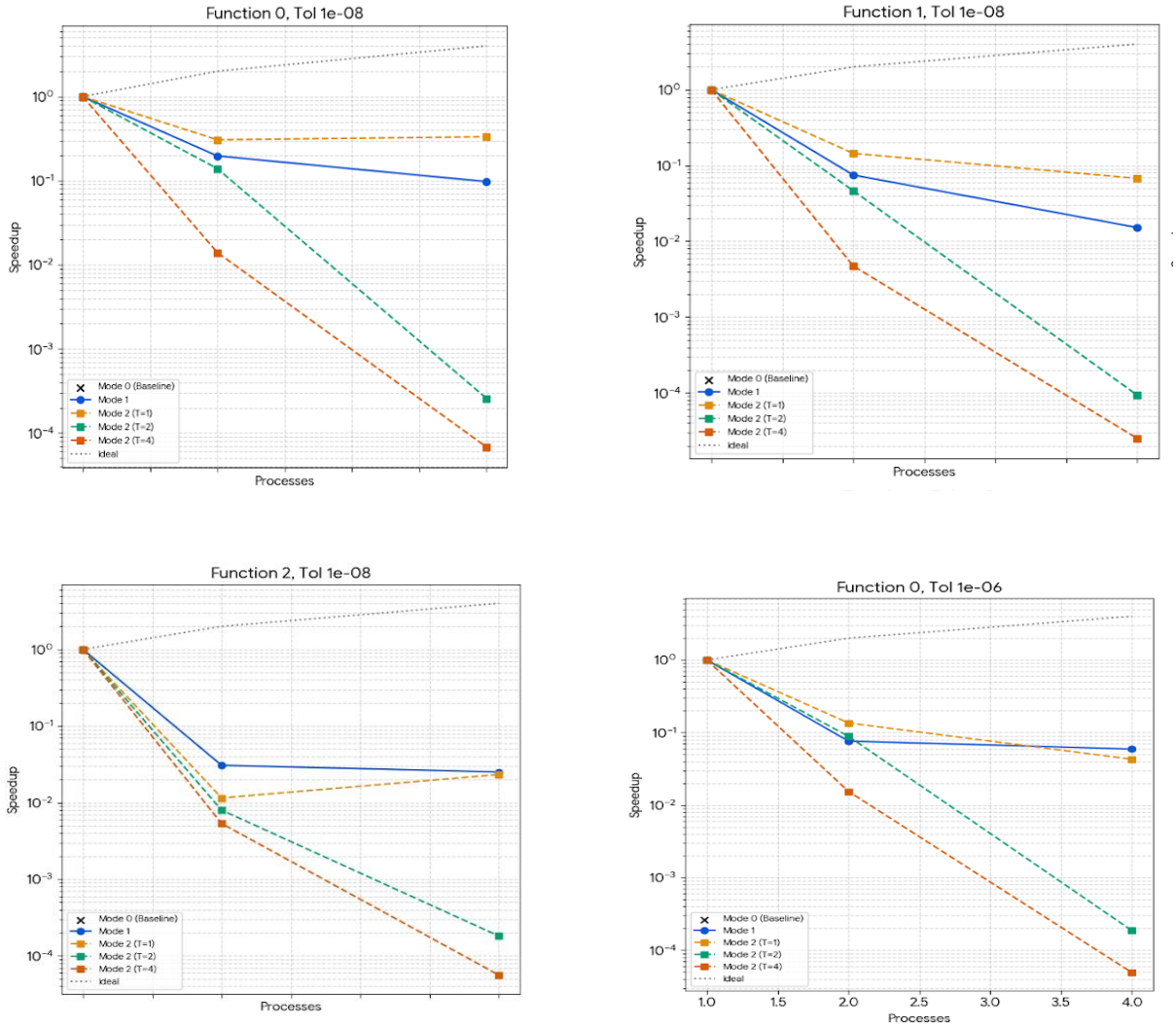
Mode	Threads	Process	Function	Tol	Time	Speedup	Efficiency
0		1	0	1e-6	0.000029	1	1
0		1	1	1e-6	0.000007	1	1
0		1	2	1e-6	0.000026	1	1
1		2	0	1e-6	0.000376	0.076151122	0.038076
1		2	1	1e-6	0.000278	0.0256	0.0128
1		2	2	1e-6	0.001615	0.015895954	0.007948
1		4	0	1e-6	0.000484	0.059187887	0.014797
1		4	1	1e-6	0.000933	0.007624494	0.001906
1		4	2	1e-6	0.000686	0.03742709	0.009357
2	1	2	0	1e-6	0.000212	0.135291033	0.067646
2	1	2	1	1e-6	0.000105	0.067724868	0.033862
2	1	2	2	1e-6	0.000600	0.042801556	0.021401
2	2	2	0	1e-6	0.000323	0.088659794	0.022165
2	2	2	1	1e-6	0.000607	0.011723759	0.002931
2	2	2	2	1e-6	0.000906	0.028322707	0.007081
2	4	2	0	1e-6	0.001860	0.015410345	0.001926
2	4	2	1	1e-6	0.002256	0.003151933	0.000394
2	4	2	2	1e-6	0.004471	0.005740699	0.000718
2	1	4	0	1e-6	0.000670	0.042807367	0.010702
2	1	4	1	1e-6	0.000149	0.047761194	0.01194
2	1	4	2	1e-6	0.000372	0.068955224	0.017239
2	2	4	0	1e-6	0.154288	0.0001858	0.000023
2	2	4	1	1e-6	0.110666	6.42574E-05	0.000008
2	2	4	2	1e-6	0.158478	0.000161958	0.000020
2	4	4	0	1e-6	0.585603	0.000049	0.000003
2	4	4	1	1e-6	0.600152	0.000012	0.000001
2	4	4	2	1e-6	0.597990	0.000043	0.000003
0		1	0	1e-8	0.000040	1	1.000000
0		1	1	1e-8	0.000015	1	1.000000
0		1	2	1e-8	0.000033	1	1.000000
1		2	0	1e-8	0.000205	0.196863169	0.098432
1		2	1	1e-8	0.000196	0.074872377	0.037436
1		2	2	1e-8	0.001071	0.030695842	0.015348
1		4	0	1e-8	0.000416	0.097326203	0.024332
1		4	1	1e-8	0.000968	0.015151515	0.003788
1		4	2	1e-8	0.001310	0.025108152	0.006277
2	1	2	0	1e-8	0.000132	0.306139613	0.153070
2	1	2	1	1e-8	0.000102	0.144420131	0.072210
2	1	2	2	1e-8	0.002860	0.011500505	0.005750
2	2	2	0	1e-8	0.000291	0.139196941	0.034799
2	2	2	1	1e-8	0.000316	0.046429828	0.011607
2	2	2	2	1e-8	0.004119	0.007984032	0.001996
2	4	2	0	1e-8	0.002888	0.014004848	0.001751
2	4	2	1	1e-8	0.003109	0.004718161	0.000590

2	4	2	2	1e-8	0.006162	0.005337661	0.000667
2	1	4	0	1e-8	0.000122	0.332723949	0.083181
2	1	4	1	1e-8	0.000217	0.067622951	0.016906
2	1	4	2	1e-8	0.001414	0.023264953	0.005816
2	2	4	0	1e-8	0.156988	0.000257628	0.000032
2	2	4	1	1e-8	0.157565	0.000093	0.000012
2	2	4	2	1e-8	0.181511	0.000181	0.000023
2	4	4	0	1e-8	0.595575	0.000068	0.000004
2	4	4	1	1e-8	0.586410	0.000025	0.000002
2	4	4	2	1e-8	0.586549	0.000056	0.000004

Hybrid configurations with large thread counts suffered severe performance degradation due to oversubscription and thread management overhead.

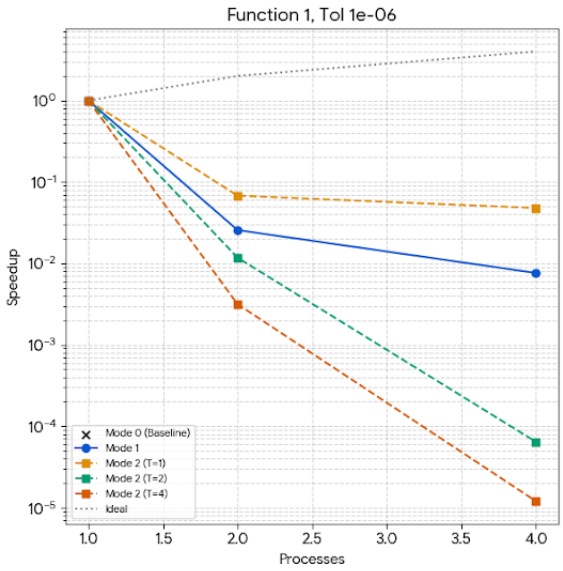
C) Plots

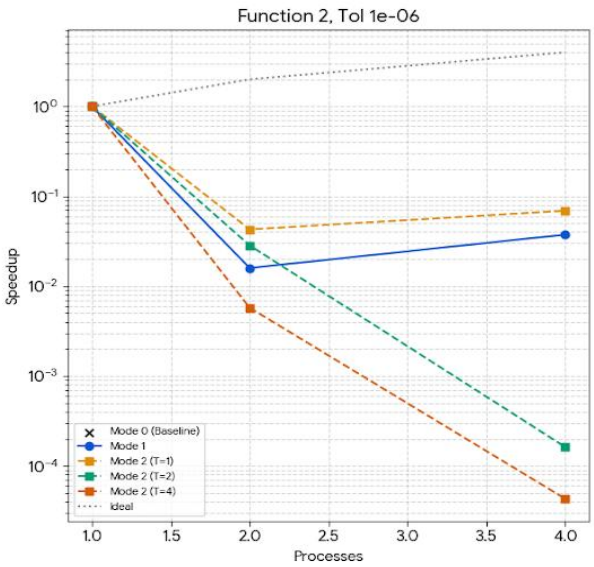
- *Speedup vs processes*



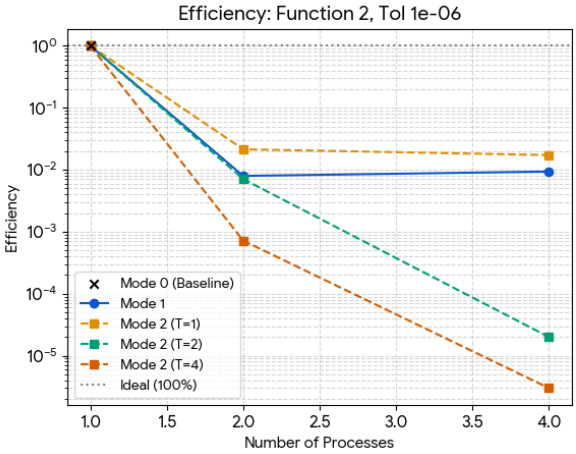
CS 6025 **Introduction Parallel Processing- Assignment 3**

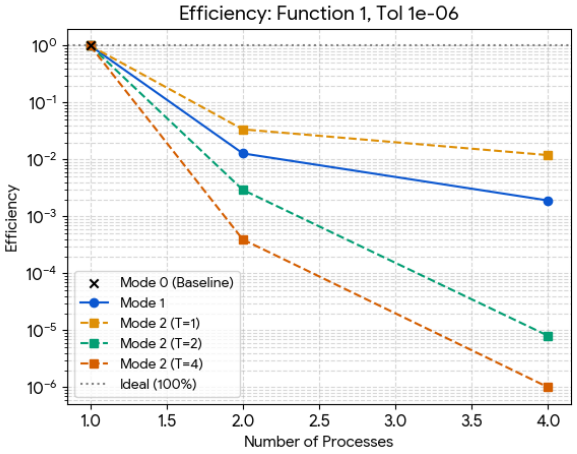
Jayashree Janani Purushothaman Student #: 3772545

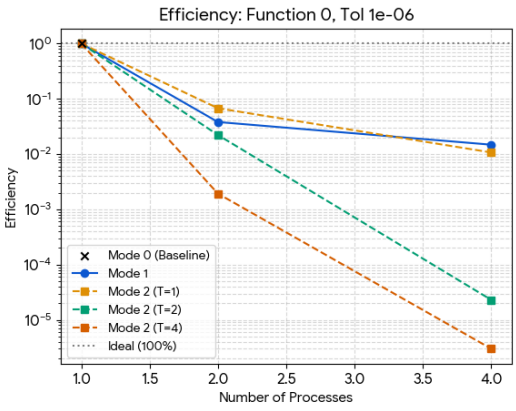


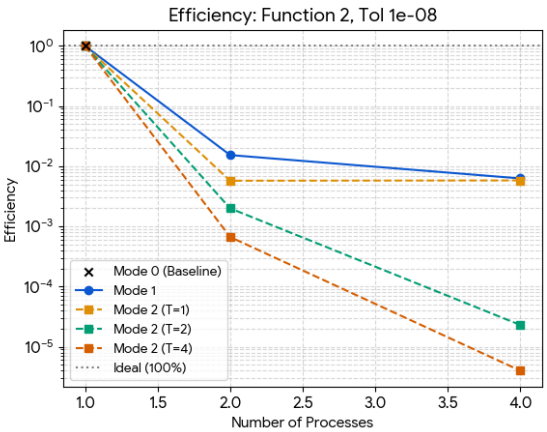


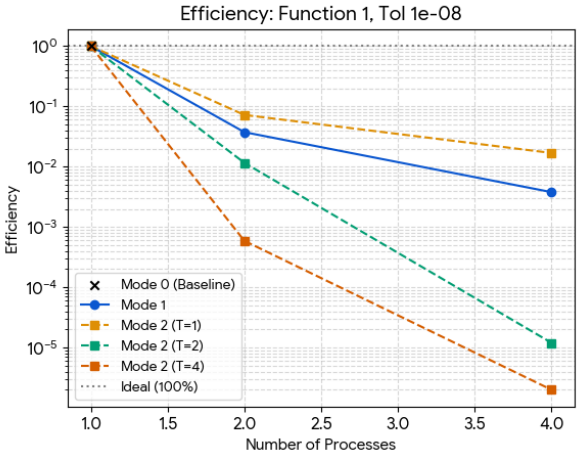
- *Efficiency vs Processes*

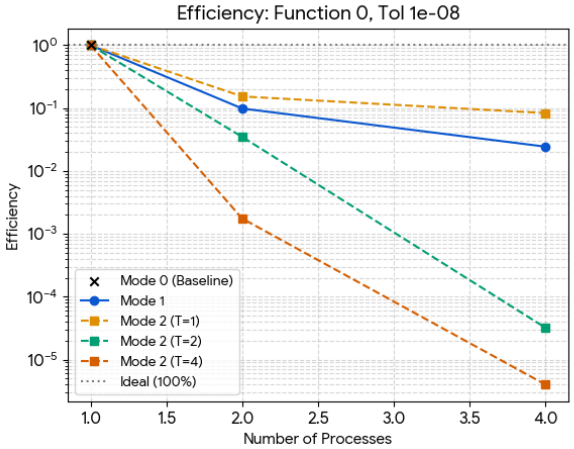












The efficiency plots show a rapid decrease in parallel efficiency as the number of processes increases for both MPI-only and hybrid implementations. This behavior is expected because the adaptive Simpson integration workload per interval is relatively small, causing communication and synchronization overhead to dominate execution time. The hybrid MPI/OpenMP implementation shows even lower efficiency when increasing thread count, indicating additional overhead from thread management during recursive interval subdivision. Efficiency improves slightly for tighter tolerances (1e−8) due to increased computational workload, which better amortizes communication costs. Functions with more irregular behavior also demonstrate reduced efficiency due to load imbalance across processes.

D) Discussion

Why does MPI dynamic perform well for irregular workloads?
Dynamic MPI redistributes work at runtime by assigning intervals on demand, which balances uneven recursive subdivisions across processes and prevents idle workers.

Why might hybrid perform worse?
Hybrid uses static coarse interval assignment, so load imbalance remains local to each process, and additional OpenMP thread-management overhead can increase runtime.

Does OpenMP compensate for lack of MPI load balancing?
No. OpenMP only balances work within a process, not across processes, so global imbalance caused by static MPI partitioning is not fully corrected.

When does hybrid outperform MPI?
Hybrid performs better when each MPI process has enough local workload and OpenMP threads can efficiently parallelize recursive subdivisions with minimal overhead.

What is the impact of thread count and process count?
Increasing processes raises communication overhead, while increasing threads raises synchronization and scheduling overhead; performance improves only when workload per process/thread is sufficiently large.

Jayashree Janani Purushothaman Student #: 3772545

E) Authorship trail

https://github.com/JayashreeJanani/Parallel---assignment-2.git

